

Assignment No: 8

Title: Competitive programming

Aim: Substring search - To solve a problem statement to demonstrate the Substring search / pattern matching.

Objective:

- Given a text string and a pattern string, find all starting indices in text where pattern appears as a substring.

CO Mapped: CO3, CO4

Problem Statement: Alice loves solving puzzles and recently found an old parchment containing a secret message. However, the message is hidden within a much longer string of gibberish. She knows the hidden message is a substring, and she wants to find **all the starting indices** where it appears in the larger string. Help Alice locate **every occurrence** of a given substring (pattern) within a longer string (text)? Use efficient algorithms such as **Knuth-Morris-Pratt (KMP)** or **Rabin-Karp** for optimal performance.

Expected Outcome:

- text = "ABABDABACDABABCABAB"
- pattern = "ABABCABAB"

Output (starting indices): [10]

Suggested Study Material:

<https://www.geeksforgeeks.org/dsa/kmp-algorithm-for-pattern-searching/>

https://en.wikipedia.org/wiki/Knuth%E2%80%93Morris%E2%80%93Pratt_algorithm

Theory / Procedure / Diagrams:

1. **Algorithms / Methods / Steps: Knuth-Morris-Pratt (KMP) Algorithm:**
 - **Core Idea:** Avoids re-comparing characters that have already been matched. It pre-processes the pattern to build a "Longest Proper Prefix Suffix" (LPS) array (also known as a "failure function" or "prefix function"). This array helps in shifting the pattern efficiently when a mismatch occurs, instead of simply shifting by one character.
 - **Time Complexity:** $O(m + n)$, where 'n' is the length of the text and 'm' is the length of the pattern. This is highly efficient.
 -
2. **Rabin-Karp Algorithm:**
 - **Core Idea:** Uses hashing to quickly compare substrings. It calculates a hash value for the pattern and then for all substrings of text of the same length as the pattern. If the hash values match, it performs a character-by-character comparison to confirm. It uses a rolling hash function to efficiently calculate hash values for subsequent substrings.
 - **Time Complexity:** Average case $O(n + m)$, worst case $O(nm)$ (rare, depends on hash collisions).

Detailed Explanation and Implementation (KMP Algorithm):

The KMP algorithm consists of two main parts:

Part 1: Preprocessing the Pattern (Building the LPS Array)

The LPS array, `lps[]`, for a pattern `P` of length `m` stores at each index `i` the length of the longest proper prefix of `P[0...i]` that is also a suffix of `P[0...i]`.

Algorithm for Building LPS Array:

1. Initialize `lps[0] = 0`.
2. Use two pointers: `length` (length of the previous longest prefix suffix) and `i` (current index).
3. Iterate `i` from 1 to `m-1`:
 - If `pattern[i] == pattern[length]`, increment `length` and set `lps[i] = length`.
 - If `pattern[i] != pattern[length]`:
 - If `length != 0`, update `length = lps[length - 1]`.
 - Else (`length == 0`), set `lps[i] = 0`.

Part 2: Searching the Text

1. Initialize two pointers: `i` for text (current character in text) and `j` for pattern (current character in pattern).
2. Iterate `i` from 0 to `n-1`:
 - If `text[i] == pattern[j]`, increment both `i` and `j`.
 - If `j` reaches `m` (length of pattern), it means a match is found. Store the starting index (`i - j`) and then update `j = lps[j - 1]` to find subsequent matches.
 - If a mismatch occurs (`text[i] != pattern[j]`):
 - If `j != 0`, set `j = lps[j - 1]`. This is the crucial step that avoids re-comparing.
 - Else (`j == 0`), simply increment `i`.

Why KMP is Efficient:

The KMP algorithm achieves $O(n + m)$ time complexity because:

- **LPS Array Construction:** The `compute_lps_array` function iterates through the pattern once, taking $O(m)$ time.
- **Searching:** The `kmp_search` function iterates through the text at most once. The `j` pointer (for the pattern) never decreases; it either stays the same or moves forward. When it shifts back using `lps[j-1]`, it's still effectively making progress relative to the initial starting point of the pattern. Each character in the text is compared with a character in the pattern at most a constant number of times. This results in $O(n)$ time for the search phase.

Therefore, the total time complexity is $O(m + n)$. This makes KMP an excellent choice for substring search problems, especially when the pattern might have repeating characters.

Test Cases:

Test Case no	Text	Pattern	Expected Output	Actual Output	Test Status
1	"applebanana"	"apple"	[0]	[0]	Pass
2	"kiwiberrypeach"	"peach"	[9]	[9]	Pass
3	"abcdefg"	"xyz"	[]	[]	Pass
4	" "	"abc"	[]	[]	Pass

Assignment No: 9

Title: Competitive programming

Aim: Suffix array construction - To solve a problem statement to demonstrate Suffix array construction.

Objective:

- To build a suffix array efficiently for strings up to length 10^5 .
- Enable fast lexicographical comparison of suffixes.
- Facilitate substring search and pattern matching in $O(m \log n)$ where m is pattern length.

CO Mapped: CO3, CO4

Problem Statement: Write a program that constructs the suffix array for the given string. A single string S of length $1 \leq |S| \leq 10^5$, consisting of **lowercase English letters**.

Expected Outcome:

Input: Single line containing string S ($1 \leq |S| \leq 10^5$).

String consists of lowercase English letters (a to z).

Ex – Banana

Output: A single line with space-separated integers - the suffix array.

Each integer represents the starting index (0-based) of a suffix in the sorted order.

Ex - Following is suffix array for banana

5 3 1 0 4 2

Suggested Study Material:

<https://daniel-hug.github.io/longest-repeated-substring/>

<http://www.stanford.edu/class/cs97si/suffix-array.pdf>

http://en.wikipedia.org/wiki/Suffix_array

Theory / Procedure / Diagrams:

A suffix array is a sorted array of all suffixes of a string, where each element in the array is the starting index of a suffix in the original string.

For example, if the string is "banana": Its suffixes are:

"banana" (starts at index 0)

"anana" (starts at index 1)

"nana" (starts at index 2)

"ana" (starts at index 3)

"na" (starts at index 4)

"a" (starts at index 5)

When sorted lexicographically, their order is:

"a" (index 5)

"ana" (index 3)

"anana" (index 1)

"banana" (index 0)

"na" (index 4)

"nana" (index 2)

So, the suffix array for "banana" is [5, 3, 1, 0, 4, 2].

Test Cases:

Test	Input	Expected Output	Actual Output	Test Status
1	banana	5 3 1 0 4 2	5 3 1 0 4 2	Pass
2	abracadabra	10 7 0 3 5 8 1 4 6 9 2	10 7 0 3 5 8 1 4 6 9 2	Pass
3	a	0	0	Pass
4	zzzzzz	5 4 3 2 1 0	5 4 3 2 1 0	Pass